

문제 해결을 위해 기술을 선택하는 백엔드 개발자 김동혁입니다.

INFO



BIRTH 2000.08.31

PHONE 010.5597.6632

MAIL kdh367036@naver.com

Education

2026.02 한성대학교 컴퓨터 공학부 학사 졸업

2020.03 한성대학교 컴퓨터 공학부 학사 입학

2019.02 상계고등학교 이과계열 졸업

2016.03 상계고등학교 입학

Certifications

2025.12 정보처리기사

2024.06 SQLD

Awards

2025.12 오픈소스 개발자 대회 장려상(한국오픈소스협회 회장상)

2025.12 카카오 테크 부트캠프 부하테스트 최우수상

2025.09 한성대학교 공학경진대회 동상

2025.03 한성대학교 캡스톤디자인 Qualcomm 기업연계 프로젝트 어플리케이션 부문 우수상

Experience

2025.09~2025.03.26 카카오테크 부트캠프(풀스택) 수료

SKILLS

Java

- JVM 구조, GC, JIT Compiler 동작 원리에 대해 이해하고 있습니다.
- Swing 기반 게임 개발 경험(멀티스레드 및 Socket 통신 적용)이 있습니다.
- 객체지향 설계 원칙(OOP)을 기반으로 클래스 구조 설계 및 구현한 경험이 있습니다.
- REST API 서버 개발 시 Spring Boot 기반 백엔드 로직을 구현한 경험이 있습니다.

Python

- LLM 기반 서비스 로직 개발에 활용한 경험이 있습니다.
- 문서에서 핵심 개념 및 관계를 추출하여 지식 그래프 생성 및 질의 응답 시스템(GraphRAG)을 구현한 경험이 있습니다.

C++

- 객체지향 설계 원칙(OOP)을 기반으로 클래스 구조 설계 및 구현한 경험이 있습니다.

SKILLS

SPRING

- Spring Boot 기반 REST API 서버를 설계 및 구현한 경험이 있습니다.
- JWT 기반 인증/인가 시스템을 구현한 경험이 있습니다.
- Spring Security 기반 Stateless 보안 구성 및 CORS/CSRF 정책을 적용한 경험이 있습니다.
- 트랜잭션 관리를 통해 데이터 정합성을 고려한 서비스를 개발한 경험이 있습니다.

FastAPI

- FastAPI 기반 LLM 서비스 API를 개발한 경험이 있습니다.
- Ollama를 활용해 지식 관리 및 QA 시스템을 구현한 경험이 있습니다.

REACT

- React + Vite를 활용해 SPA를 개발한 경험이 있습니다.

SKILLS



MySQL

- PK/FK 기반 관계형 스키마를 설계하고 인덱스를 적용한 경험이 있습니다.
- AUTO_INCREMENT 기반 PK 생성 방식을 이해하고 활용한 경험이 있습니다.



PostgreSQL

- PK/FK 기반 관계형 스키마를 설계하고 인덱스를 적용한 경험이 있습니다.
- PostgreSQL의 Sequence 기반 PK 생성 방식을 이해하고 활용한 경험이 있습니다.



REDIS

- Redis Pub/Sub 기반 채팅 메시지 이벤트 전파 구조 구현 경험이 있습니다.
- TTL 기반 캐싱 및 토큰 만료 관리 적용 경험이 있습니다.
- Redis 캐시를 활용한 조회 성능 최적화 경험이 있습니다.
- Redis Streams 기반 비동기 메시지 처리 구현 경험이 있습니다.

PROJECT

팀 프로젝트 / 모여밥	
기간	2025.12.22~
팀원	총 6명(Full-Stack 1명,Backend 1명, DevOps 2명, AI 2명)
담당 역할	Backend
기술 스택	Spring Boot, React, FastAPI, PostgreSQL, Redis, RabbitMQ

팀 프로젝트 / BrianTrace	
기간	2025.02 ~2025.10
팀원	총 4명(Frontend 1명, Backend/AI 1명, DevOps 1명, AI 1명)
담당 역할	Backend/AI
기술 스택	FastAPI, Neo4J, SQLite, Qdrant, Ollama

개인 프로젝트 / TrueDev	
기간	2025.11.24~2025.12.9
팀원	총 1명
담당 역할	Full-Stack
기술 스택	Spring Boot, React, FastAPI, MySQL, Redis, Ollama

팀 프로젝트

모여밥

Roll Backend

SKILLS Spring Boot, PostgreSQL, MongoDB, Redis, RabbitMQ

Backend GitHub(Click here)

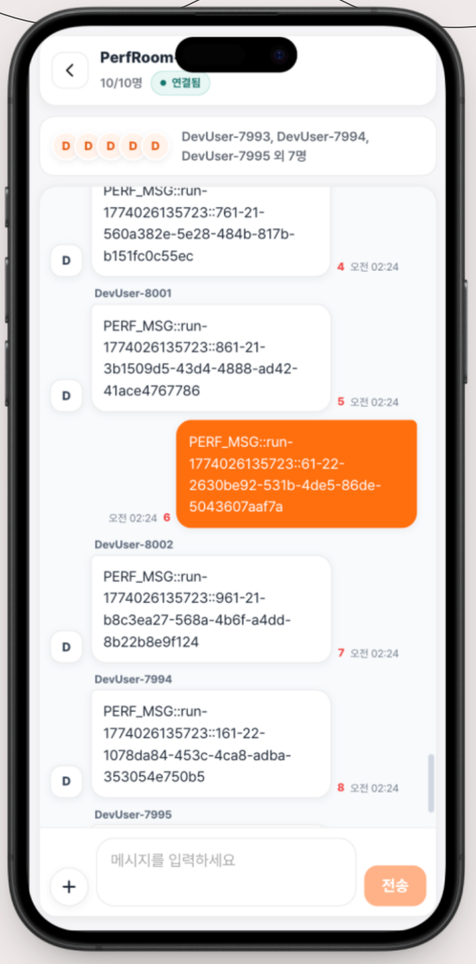


모여밥은 친구 모임, 회식, 뒤풀이에서 반복되는 식당 결정의 불편함을
해결하는 그룹 식사 추천 서비스입니다.
참여자의 취향과 거리, 리뷰를 AI가 종합 분석해 최적의 장소를 추천하고,
스와이프 투표를 통해 빠른 의사결정을 돕습니다.

실시간 단체 채팅 시스템

부하 테스트 환경

항목	값
채팅방	100개
방당 참여자	10명
동시 접속	1,000명
STOMP 메시지	1,000 msg/s (방당 10 msg/s)
GET /messages	150 req/s (채팅방 입장 시 조회)
POST /read-pointer	50 req/s (읽음 처리)
테스트 시간	10분
기존 채팅 데이터	~800,000건



지표 해석 참고

	k6 (클라이언트 측정)	Prometheus (서버 측정)
측정 구간	클라이언트가 요청을 보낸 순간 ~응답을 받은 순간	서버가 요청을 처리하기 시작한 순간 ~응답을 보낸 순간
포함 시간	네트워크 + Tomcat 큐 대기 + 서버 처리	서버 처리만
집계 범위	GET+POST 요청 합산	엔드포인트 별 분리

실시간 단체 채팅 시스템

문제

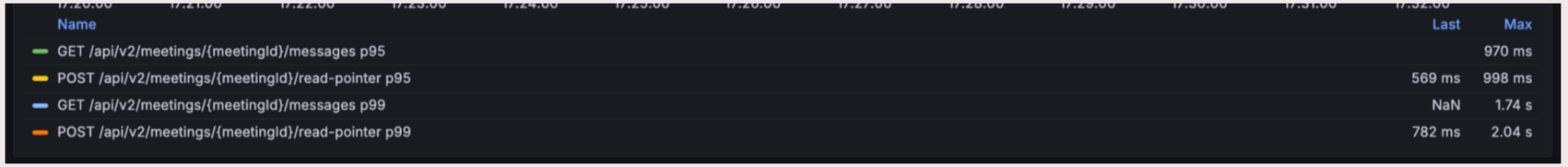
DB Pool(Hikari)

- 커넥션 풀 max 20개에서 pending이 200까지 반복 급증
- Tomcat 스레드(200개)가 모두 커넥션 획득 대기 상태로 블로킹되어 스레드 풀이 고갈되고, 신규 요청을 처리할 수 없는 상태



API Latency

- GET /messages p99가 테스트 시작 후 200ms → 1.74s로 지속 악화
- POST /read-pointer p99가 최대 2.04s까지 상승



실시간 단체 채팅 시스템

원인

HikariCP 커넥션 풀 병목

사용자의 모든 요청이 단일 커넥션 풀(max 20)을 두고 경쟁

요청 유형	PostgreSQL 연산	초당 호출량
GET /messages	3-way JOIN (Message + Participant + Member)	~150 req/s
POST /read-pointer	UPDATE (last_read_id 갱신)	~50 req/s
메시지 저장 (STOMP)	INSERT 1건 + UPDATE 2건	~1,000 msg/s
unread count 갱신	SELECT (window) + Redis broadcast	매 메시지마다
합계		~1,200+ DB 연산/s

핵심

개별 쿼리는 수 ms로 빠르게 수행되었으나, 커넥션 획득 대기로 인해 수백 ms에서 수 초의 지연이 발생하며 병목이 발생했습니다.

CPU는 5~20% 수준으로 여유가 있었지만, 스레드가 커넥션 풀에서 블로킹되며 전체 응답 성능이 저하되었습니다.

커넥션 풀 또한 늘려봤지만, 이는 포화 시점을 늦출 뿐 채팅의 높은 연산 볼륨이 풀 공급을 초과하는 근본 문제를 해소하지 못했습니다.

실시간 단체 채팅 시스템

개선 1. Read Pointer no-op write 제거

- 문제: 읽음 포인터(last_read_id)가 이미 최신값인데도 매번 UPDATE 쿼리를 실행하여 불필요한 DB 쓰기 발생
- 해결: last_read_id가 실제로 전진할 때만 UPDATE, 동일하거나 후퇴하는 경우 스킵
- 효과: POST /readPointer p95 **2,421ms → 1,736ms (-28.3%)**

개선 2. Unread Count Refresh Coalescing

- 문제: 메시지 도착마다 unread count 재계산 + broadcast 가 매번 발생하여 부하 집중
- 해결: 동일 meeting에 대한 unread 갱신 요청을 일정 시간 window 내에서 병합하여 한 번에 처리
- 효과: unread 재계산·broadcast 호출 51.2% 감소 (초당 1,000회 → 488회)

개선 3. 최근 메시지 Redis 캐시

- 문제: GET /messages 호출마다 PostgreSQL에서 3-way JOIN 실행
- 해결: Redis에 최근 메시지를 캐싱하고, version key 기반 무효화 전략을 적용해 최신성 확보
- **캐시 히트 98.7%**를 달성하고, 전반적인 응답 지연과 요청 드롭 개선

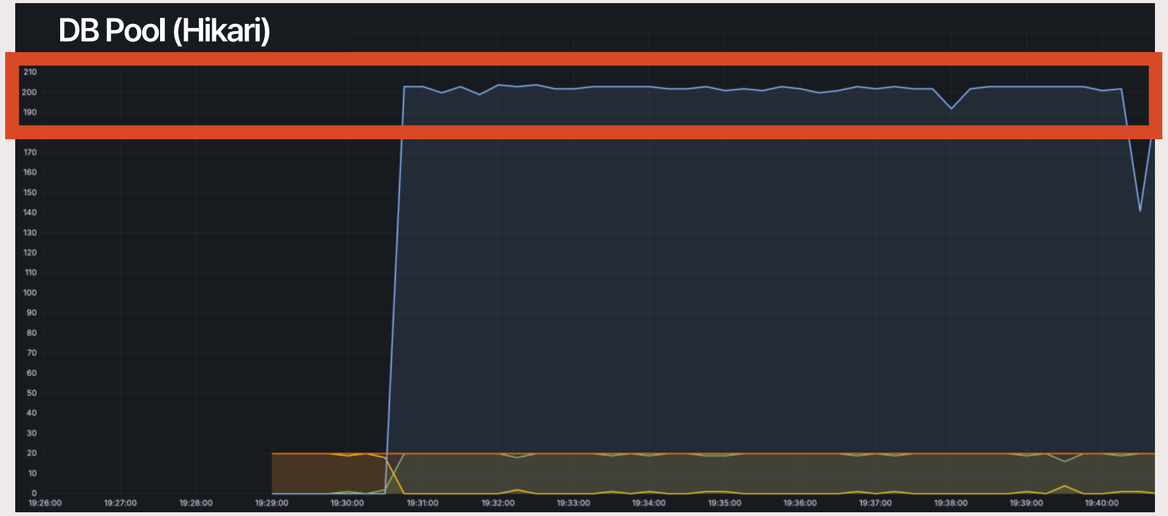
실시간 단체 채팅 시스템

결과

지표(클라이언트 측정)	Before	After	개선율
응답 시간 p95	8,149ms	3,877ms	-52.4%
응답 시간 평균	3,509ms	1,748ms	-50.2%
처리량 (req/s)	138	184	+33.6%
요청 드롭 수	22,429	5,610	-75.0%

한계점

- DB 연산 횟수 감소로 체감 응답 시간은 개선되었으나, 여전히 성능은 좋지 않음
- 서버 처리 시간도 소폭 개선되었으나 근본적인 커넥션 풀 포화는 해소되지 않음



실시간 단체 채팅 시스템

병목 분석

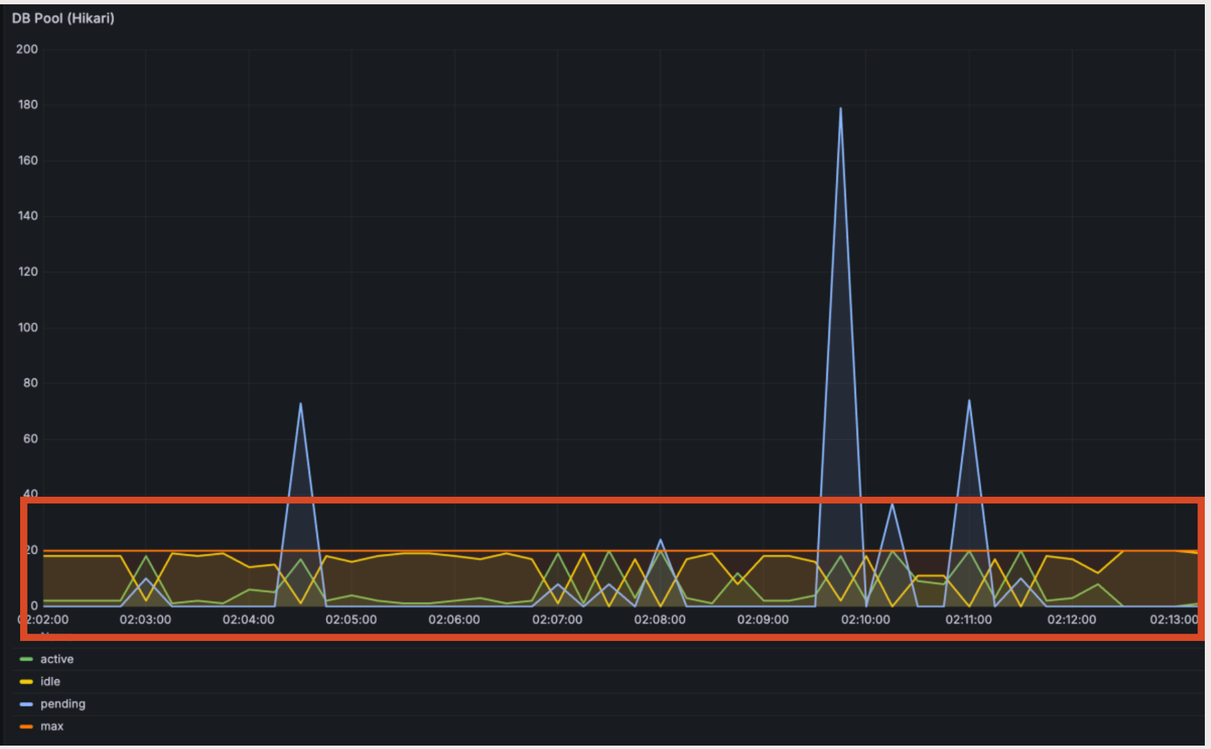
- 서버 처리 시간은 개선 이후 소폭 감소 → 병목은 쿼리가 아니라 커넥션 대기 때문
- 채팅 트래픽이 서비스 DB와 커넥션 풀을 공유하는 구조가 원인
- **채팅 저장소 분리 필요**

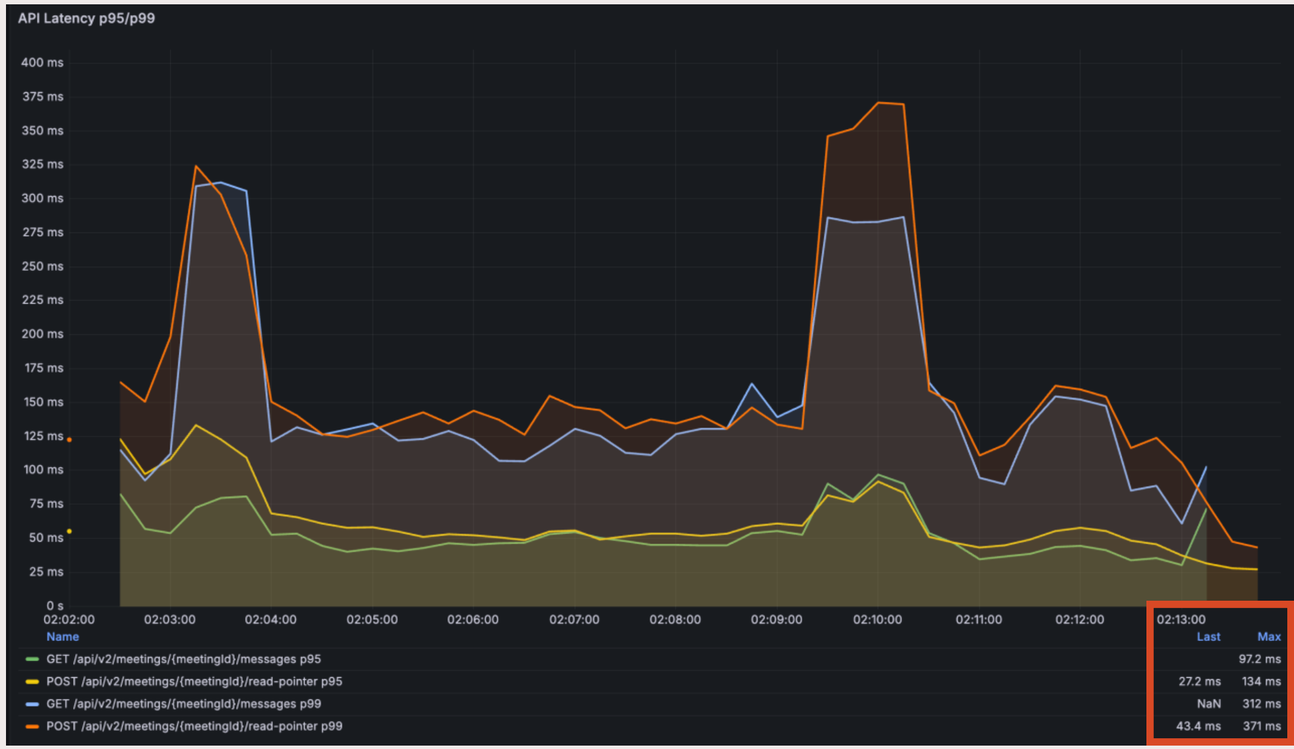
PostgreSQL 인스턴스 추가 VS 몽고 DB 마이그레이션

- PostgreSQL 인스턴스 추가로 커넥션 분산은 가능했으나, 트래픽 증가 시 동일한 병목이 재발할 수 있는 구조적 한계 존재
- 채팅 서비스 특성상 **INSERT 중심** 워크로드로, RDB의 트랜잭션 및 락 오버헤드가 상대적으로 비효율적으로 작용
- 조회 패턴이 최신 메시지 역순 조회로 단순하여, **JOIN 및 RDB**의 이점을 활용하기 어려운 구조
- 채팅방 기준으로 샤딩을 고려할 때, PostgreSQL 대비 MongoDB의 **수평 확장 구조**가 더 적합하다고 판단
- 단기적인 커넥션 병목 완화가 아닌, 워크로드 특성에 적합한 구조적 개선을 위해 **MongoDB로 마이그레이션 결정**

실시간 단체 채팅 시스템

- 몽고 DB 마이그레이션 이후의 그라파나 대시보드





- pending이 대부분 0 — 간헐적 스파이크만 발생하고 즉시 해소
- active 커넥션도 max(20) 이하에서 안정적으로 유지

- 부하 집중 구간에서도 300ms 내외로 유지 -> tail latency 문제 해소

실시간 단체 채팅 시스템

핵심 KPI

- k6 전체 HTTP 요청(GET /messages + POST /read-pointer) 합산 기준
- 시드 데이터: 800,000

구분	지표	PostgreSQL 개선 전	PostgreSQL 최적화	MongoDB 전환	총 개선율
응답 지연	p95	8,149ms	3,877ms	92ms	**-98.9%**
응답 지연	평균	3,509ms	1,748ms	21ms	**-99.4%**
처리량	req/s	138	184	201	**+45.7%**
안정성	요청 드롭 수	22,429	5,610	6	**-99.97%**

실시간 단체 채팅 시스템

(Redis 캐싱 전략)

배경

- 매번 DB(MongoDB)에서 최근 메시지를 조회하면 불필요한 부하 발생
- Redis에 최근 메시지 100건을 캐싱하여 DB 조회를 최소화

Version Key 기반 캐시 무효화

- 캐시 키에 버전 번호를 포함하고, 새 메시지 도착 시 새 버전을 먼저 생성한 후 이전 버전을 삭제
- 삭제와 재적재 사이의 공백이 없기에 **cache stampede**를 방지
- 다중 인스턴스 환경에서의 동시 갱신은 **Redis 분산 락**으로 제어

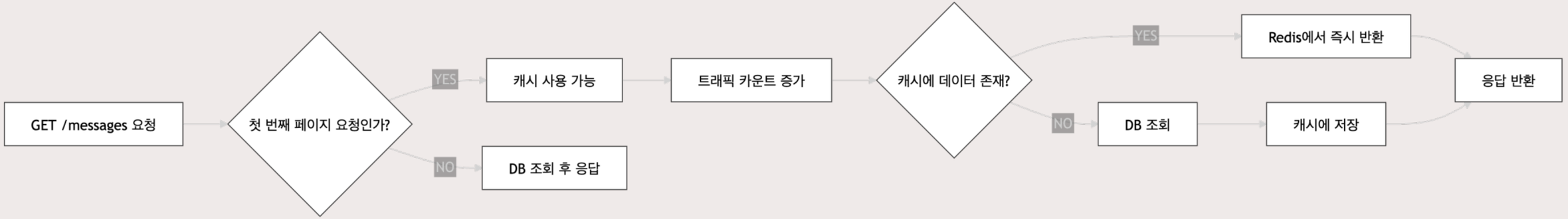
Redis Key 구조

chat:meeting:messages:**version**:{meetingId} → **현재 버전 번호** (예: 42)
chat:meeting:messages:**recent**:{meetingId}:v:42 → **최근 메시지 100건** (Redis List)
chat:meeting:messages:**traffic**:{meetingId} → **60초간 조회 횟수** (트래픽 카운터)
chat:meeting:messages:**lock**:{meetingId} → **분산 락** (SETNX)
chat:meeting:**latest-message-id**:{meetingId} → **최신 메시지 ID**

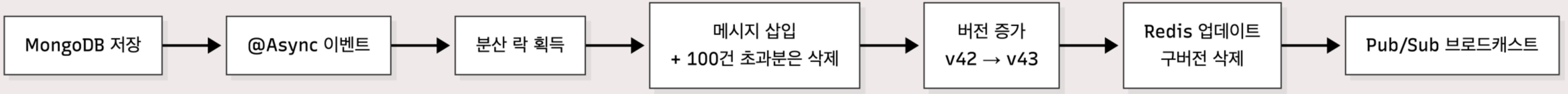
실시간 단체 채팅 시스템

(Redis 캐싱 전략)

캐시 조회 흐름



캐시 갱신 흐름



실시간 단체 채팅 시스템

(Hot/Cold Data 차등 TTL 전략)

배경

- 모든 채팅방을 동일하게 캐싱하면 비활성 채팅방까지 Redis 메모리를 점유
- **트래픽 기반**으로 캐싱 대상을 선별하고, **활성도에 따라 TTL을 차등** 적용

활성 채팅방 vs 비활성 채팅방

	활성 채팅방	비활성 채팅방
기준	60초간 채팅방 입장 10회 이상	60초간 채팅방 입장 10회 미만
TTL	60초	300초
Why?	실시간 메시지는 WebSocket으로 처리하고, 채팅방 진입 시 초기 데이터는 GET 요청으로 조회. 사용자 재진입이 빈번한 특성을 고려해 첫 메시지 조회에 캐싱을 적용하여 응답 속도를 개선.	접근 빈도가 낮은 채팅방에 대해서는 Cold Data로 분류하고 TTL을 늘려, 불필요한 DB 재조회 발생을 줄임

실시간 단체 채팅 시스템

(주요 인덱싱)

MongoDB: 커서 기반 페이지네이션

- 인덱스: { meetingId: 1, _id: -1 } (meetingId 오름차순, _id 내림차순=최신순)
- 마지막으로 조회한 메시지의 ObjectId를 커서로 사용하여 이후 메시지만 조회
- ObjectId가 타임스탬프 기반이라 삽입 순서 = 정렬 순서 → 별도 시퀀스 불필요
- offset 방식(SKIP N)은 데이터가 많아질수록 느려지지만, 커서 방식은 항상 O(log N)
- meetingId 필터 + _id 정렬을 인덱스만으로 처리 (Covered Query)

PostgreSQL: Partial Index

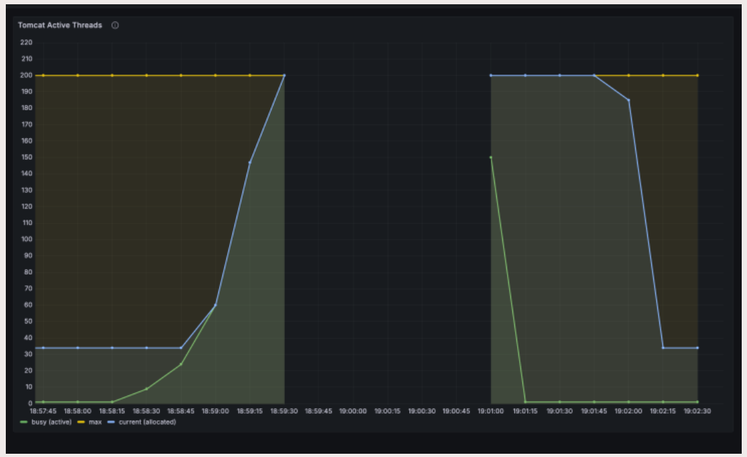
- 인덱스: (meeting_id, member_id) WHERE status = 'ACTIVE'
- 채팅 API에서 참여자 조회는 항상 status = 'ACTIVE' 조건이 붙음
- 기존 인덱스는 탈퇴/비활성 참여자까지 전부 포함 → 불필요한 데이터가 인덱스에 존재
- Partial Index로 ACTIVE 참여자만 인덱스에 포함시켜 인덱스 크기 감소 및 탐색 성능 향상

챗봇 API 호출 안정화 전략

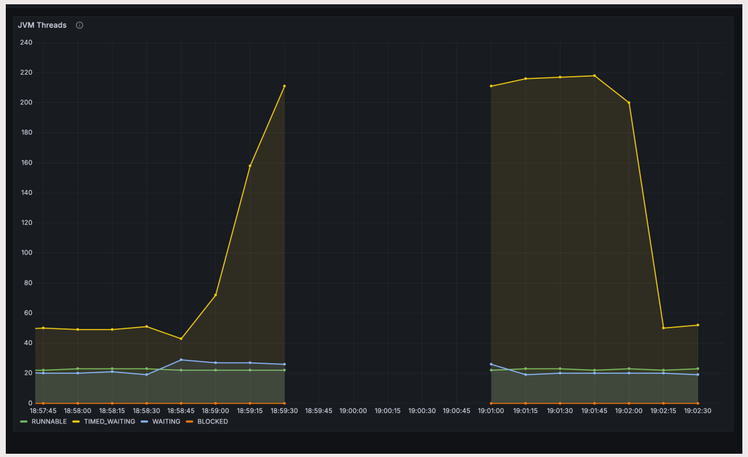
문제

- 부하 테스트를 통해 AI 서버(RAG 챗봇)를 호출하는 챗봇 API에서, 스레드 블로킹 → 스레드풀 고갈 확인

Tomcat Active Threads



JVM Threads



부하 조건: 초당 10건 요청, RAG API 응답 시간은 3~5초

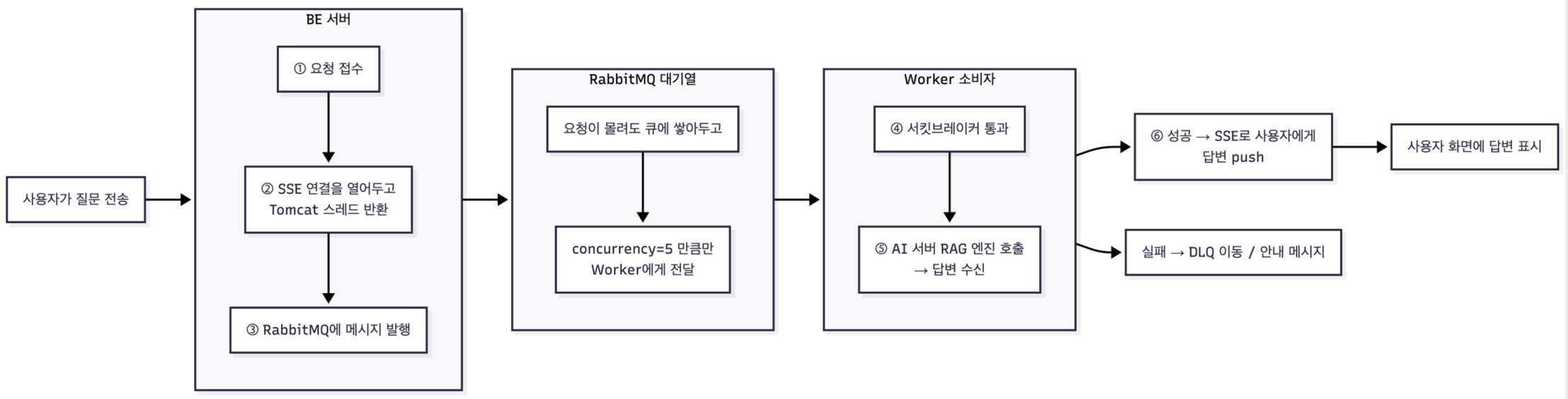
- Tomcat 스레드 200/200 포화
- TIMED_WAITING 스레드 포화
- 답변 성공률 14.8%

챗봇 API 호출 안정화 전략

해결

- Queue concurrency 설정→ AI 서버 동시 호출 수 제한
- 서킷브레이커 → AI 서버 지연/장애 시 추가 호출 차단, 장애 전파 방지
- RabbitMQ + SSE 적용 → 비동기 처리로 전환

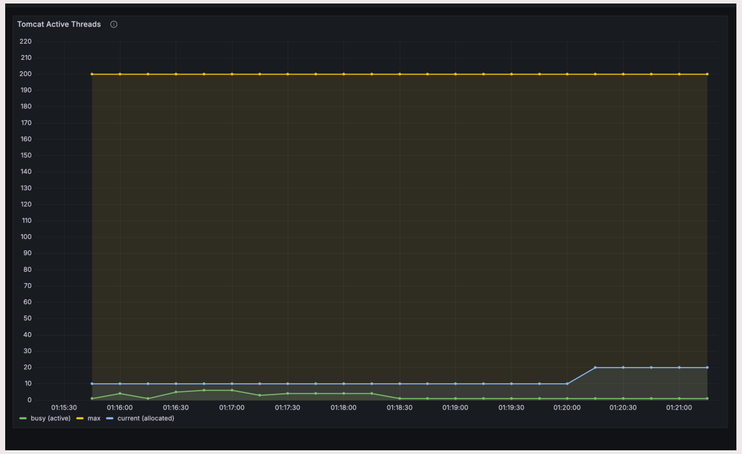
개선 이후 API 호출 흐름



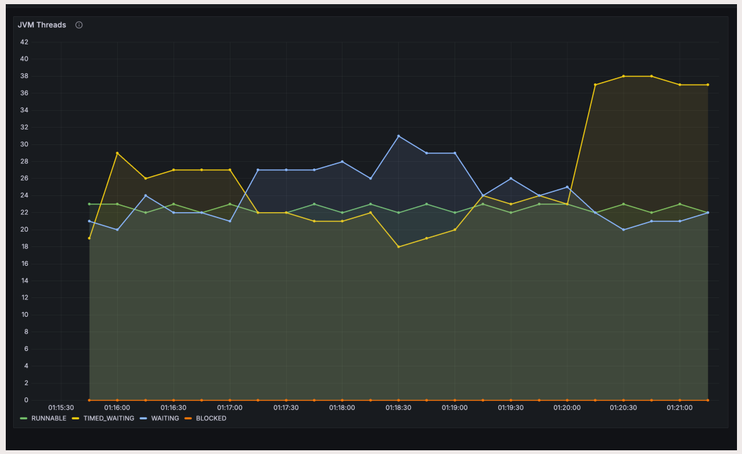
챗봇 API 호출 안정화 전략

결과

Tomcat Active Threads



JVM Threads



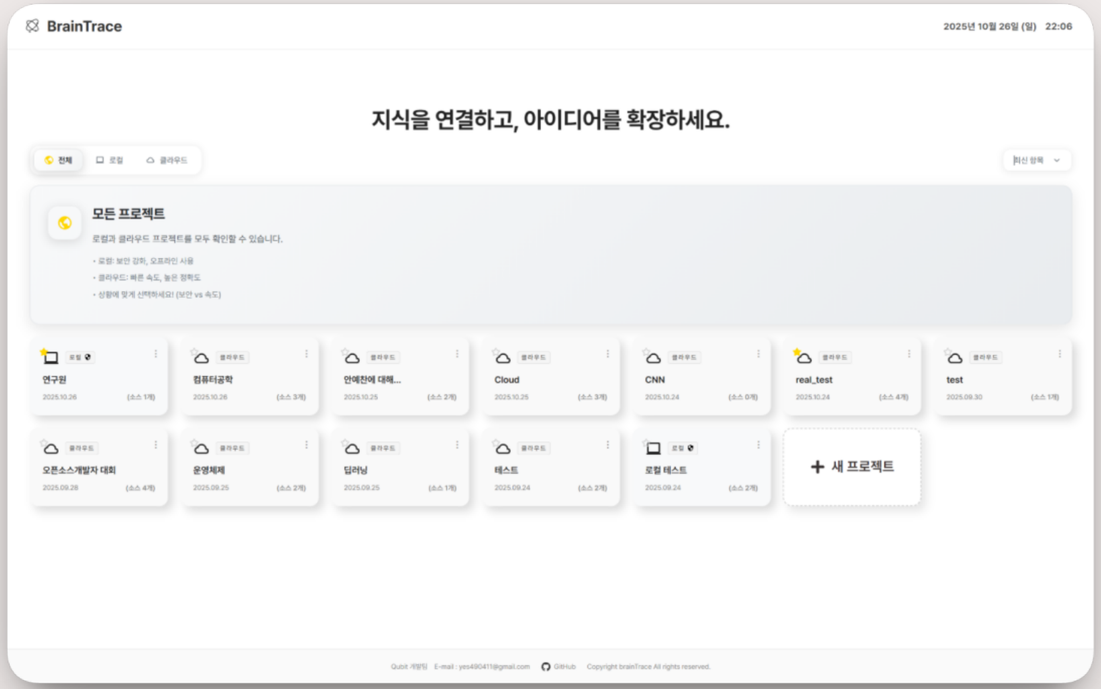
부하 조건: 초당 10건 요청, RAG API 응답 시간은 3~5초

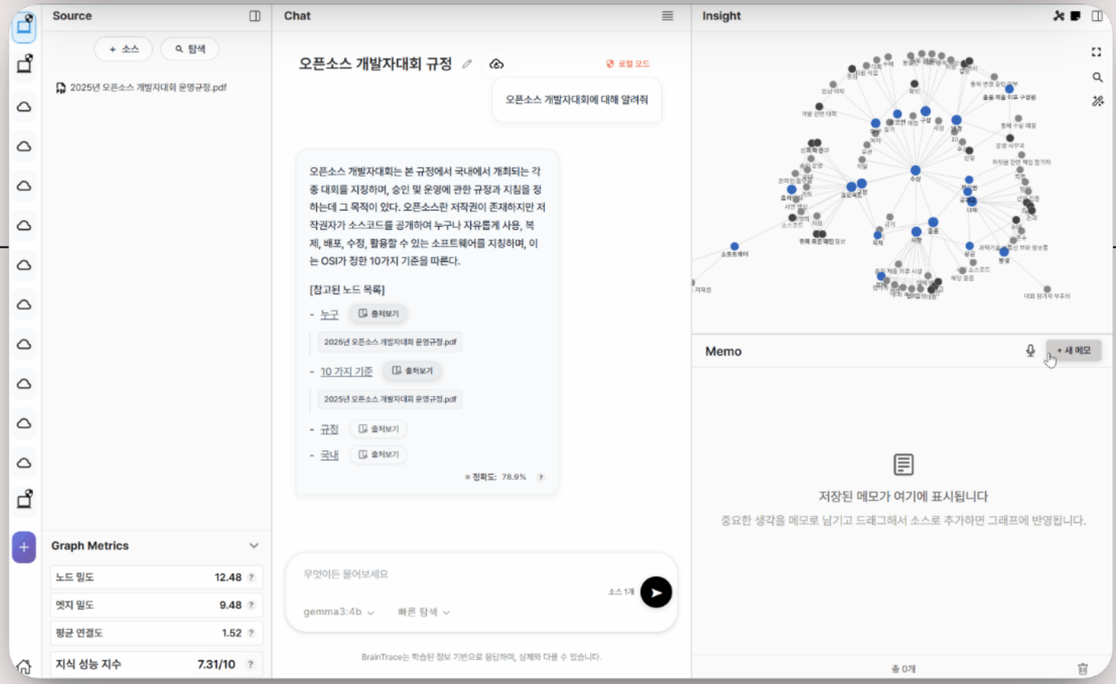
- Tomcat 스레드 안정
- TIMED_WAITING 스레드 안정
- 요청 유실 0건

지표	개선 전	개선 후
Tomcat 스레드	200/200 포화	10 이하 안정
TIMED_WAITING 스레드	~220 (포화)	~20 (안정)
요청 유실	116건	0건

BrainTrace

Roll	Backend/AI
SKILLS	FastAPI, SQLite, Qdrant, Neo4j, Ollama
시연 영상	YouTube(Click here)
GitHub	GitHub(Click here)
관련 블로그	지식 그래프 생성 및 QA 시스템 구현에 대한 블로그 포스트(Click here)





PDF, TXT, DOCX, Markdown 문서에서 핵심 개념과 관계를 자동으로 추출하여 지식 그래프로 구조화하는 서비스입니다.
사용자가 질문을 입력하면 단순 키워드 매칭이 아닌, 그래프 탐색 기반으로 개념 간 연결을 따라가며 문서 기반 답변을 생성합니다.
로컬과 클라우드 환경을 모두 지원하며, 외부 서버 없이 독립 실행이 가능해 보안에 민감한 환경에서도 활용할 수 있습니다.

BrainTrace - AI Stack

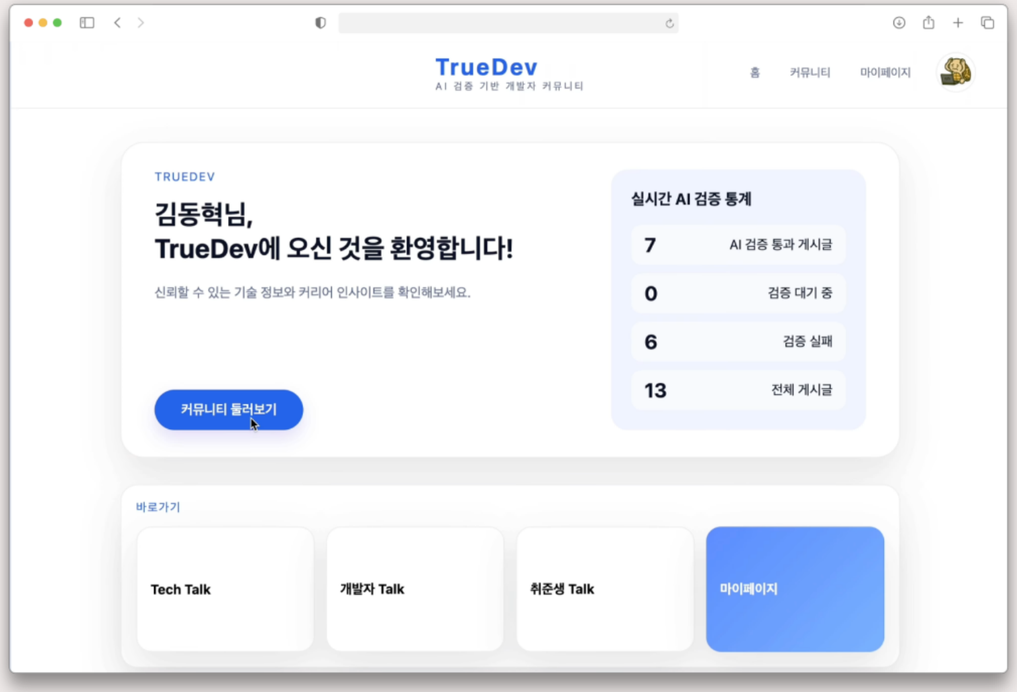
- **Qdrant**
  - Qdrant 노드&소스 데이터 임베딩후 저장시 사용
- **Neo4J**
  - Neo4J 노드간 관계를 저장하는데 활용 (지식그래프로 활용하기 위해)
- **Ollama**
  - Ollama 로컬 환경에서 LLM을 실행하기 위해 사용
- **e5-large-korean**
  - e5-large-korean 한국어로 파인튜닝된 임베딩 모델
- **Gamma3**
  - Gamma3: 4b 로컬환경 llm으로 답변 및 지식그래프 생성시 활용
- **Examine3.5**
  - Examine3.5 2.4B 답변시 활용
- **large-v3openai/whisper-large-v3**
  - large-v3openai/whisper-large-v3 음성인식 후 소스로 변환하는데 활용

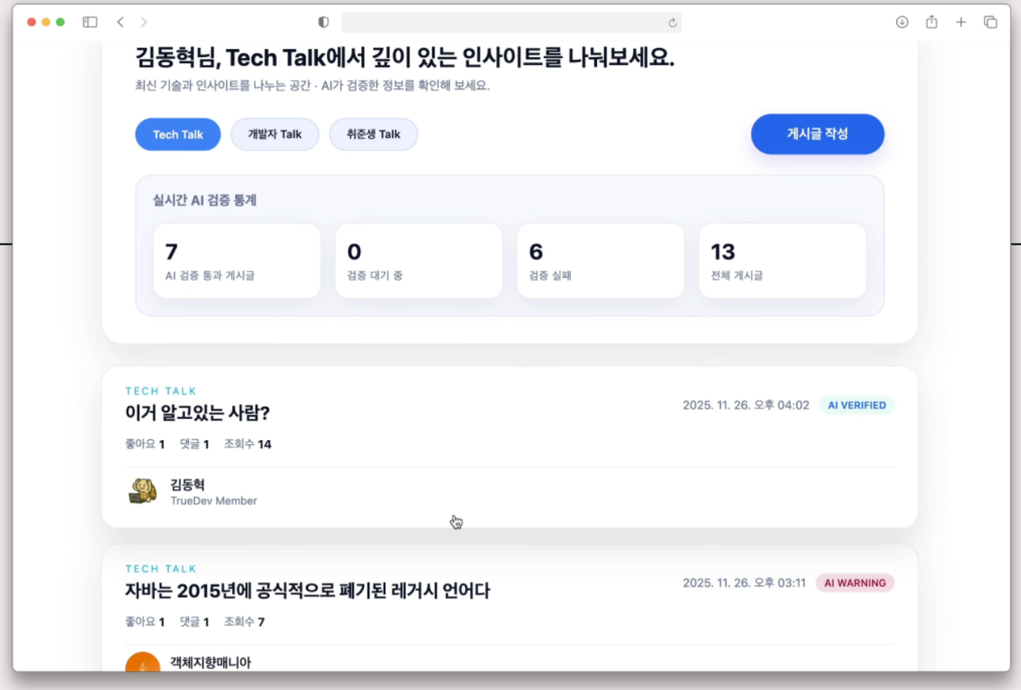
개인 프로젝트

TrueDev

Roll	Full-Stack
SKILLS	Spring Boot, React, FastAPI, MySQL, Redis, Ollama
시연 영상	YouTube(Click here)
Backend	GitHub(Click here)
Frontend	GitHub(Click here)

가짜 정보 없는 개발자 커뮤니티를 목표로,
로컬 LLM을 활용해 게시글과 댓글의 내용을 검증하고
신뢰도 높은 정보를 공유할 수 있도록 돕는 서비스입니다.





DB 커넥션 고갈 문제

문제

게시글 AI 검증을 동기 방식으로 처리하면서 LLM 응답을 기다리는 동안 요청 스레드와 DB 커넥션이 점유.
JMETER로 초당 100 요청(100 REQ/S) 부하 테스트를 수행하자 검증 요청이 동시에 몰리면서 **커넥션 풀이 고갈**되고 API 요청이 TIMEOUT으로 실패하는 문제가 발생.

```
: SQL Error: 0, SQLState: null
: HikariPool-1 - Connection is not available, request timed out after 30004ms (total=10, active=10, idle=0, waiting=51)
: REQ POST /articles/5/verify user=1 status=500 durationMs=30005
: SQL Error: 0, SQLState: null
: HikariPool-1 - Connection is not available, request timed out after 30004ms (total=10, active=10, idle=0, waiting=50)
: REQ POST /articles/5/verify user=1 status=500 durationMs=30005
: SQL Error: 0, SQLState: null
: HikariPool-1 - Connection is not available, request timed out after 30004ms (total=10, active=10, idle=0, waiting=49)
: REQ POST /articles/5/verify user=1 status=500 durationMs=30006
: SQL Error: 0, SQLState: null
: HikariPool-1 - Connection is not available, request timed out after 30003ms (total=10, active=10, idle=0, waiting=48)
: REQ POST /articles/5/verify user=1 status=500 durationMs=30004
: SQL Error: 0, SQLState: null
: HikariPool-1 - Connection is not available, request timed out after 30004ms (total=10, active=10, idle=0, waiting=47)
: REQ POST /articles/5/verify user=1 status=500 durationMs=30005
: SQL Error: 0, SQLState: null
```

DB 커넥션 고갈 문제

해결

해당 문제를 해결하기 위해 다음과 같은 방안을 검토.

- **커넥션 풀 확장** → 근본적인 해결이 아니며, 트래픽 증가 시 동일한 문제가 발생
- **트랜잭션 분리** → 코드 복잡도 증가 및 부분 실패/재시도 처리 필요
- **메시지 큐 도입** → 요청 처리와 LLM 호출을 분리 가능

최종적으로 Redis Streams를 활용해 메시지 큐를 구성하고, 검증 요청을 즉시 처리하지 않고 큐에 적재한 뒤 Worker가 순차적으로 LLM 기반 검증을 수행하도록 **비동기 처리 구조**로 변경.
또한 검증 완료 시 **SSE 이벤트**를 통해 사용자에게 전달하도록 구성.

결과

요청 스레드와 DB 커넥션 점유 시간이 감소했으며, 커넥션 풀 고갈로 인한 **API timeout** 문제를 해결.

조회수 처리 병목 문제

문제

기존에는 게시글 상세 조회와 조회수 증가가 **하나의 메서드에서 함께 실행**되는 구조.
이로 인해 **읽기 요청에도 DB write 작업이 포함**되어 응답 경로가 길어지고 성능 저하가 발생할 수 있는 구조.
상세 조회 시 좋아요 수와 댓글 수를 **count 쿼리**로 계산하고 있어 불필요한 DB 조회가 발생.

조회수 증가 전략 비교 (JMeter 500 req/s)

조회수 증가 로직 분리 후 동기 처리 (원자적 UPDATE) - **느림**

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	2500	751	8	2676	415.19	0.00%	507.8/sec	368.65	171.09	743.4
TOTAL	2500	751	8	2676	415.19	0.00%	507.8/sec	368.65	171.09	743.4



조회수 증가 로직 분리 후 비동기 처리 (Async + 원자적 UPDATE) - **빠름**

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	2500	281	4	1426	188.27	0.00%	1008.1/sec	731.44	339.63	743.0
TOTAL	2500	281	4	1426	188.27	0.00%	1008.1/sec	731.44	339.63	743.0



비관적 락으로 조회수 증가 (배타락) - **매우 느림**

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	2500	1400	27	3061	420.92	0.00%	305.5/sec	221.95	102.94	743.8
TOTAL	2500	1400	27	3061	420.92	0.00%	305.5/sec	221.95	102.94	743.8



조회수 처리 병목 문제

⟋ 해결

기존에는 게시글 상세 조회와 조회수 증가 로직이 하나의 메서드에서 함께 실행되어, 읽기 요청에도 DB write 작업이 포함되는 구조.
이로 인해 응답 경로가 길어지고 성능 저하가 발생할 수 있었으며, 좋아요·댓글 수를 count 쿼리로 계산하면서 불필요한 DB 조회도 발생.
이를 개선하기 위해 조회수 증가 로직을 분리하고 구조를 개선한 뒤, 조회수 증가 방식을 동기 처리와 비동기 처리로 나누어 성능을 비교.
또한 조회수는 실시간 정합성이 필수적인 값이 아니라고 판단하여 비동기 처리 방식을 선택.
이에 따라 다음과 같이 구조를 개선.

- 게시글 상세 조회와 조회수 증가 **로직 분리**
- 상세 조회에 **@Transactional(readOnly = true)** 적용
- 좋아요·댓글 수를 count 쿼리 대신 Article **집계 컬럼**으로 처리
- 조회수 증가를 **@Async** 기반 비동기 메서드로 분리
- view_count = view_count + 1 형태의 **원자적 UPDATE** 쿼리 적용

이를 통해 요청 응답 경로에서 조회수 증가 로직을 분리하여 응답 성능을 개선했으며, 응답 시점과 DB 반영 시점 간 일시적인 차이는 발생하지만 최종적으로는 데이터 정합성을 유지하도록 설계.

⟋ 결과

조회수 증가 전략 비교 결과, **비동기 처리 방식**은 개선된 동기 처리 방식 대비 평균 응답시간 약 **63% 감소(751ms → 281ms)**,
처리량은 약 **2배 상승(507 req/s → 1008 req/s)**.

JWT 인증/인가 구축

⟋ 배경

커뮤니티 API는 인증 여부에 따라 접근 권한이 달라져, 세션 없이 동작하는 **Stateless 인증/인가 체계**가 필요.
또한 Access Token 만료 후 UX 저하를 줄이고, 로그아웃 및 토큰 탈취 상황에서 서버 측 제어가 가능한 구조가 필요.

⟋ 구현

- Spring Security 기반으로 **SecurityFilterChain**을 구성하고, 세션을 사용하지 않는 STATELESS 인증 구조로 설계.
- 공개 API와 인증이 필요한 API를 라우팅 레벨에서 분리하고, 나머지 요청은 authenticated()로 보호.
- JwtFilter를 UsernamePasswordAuthenticationFilter 이전에 삽입하여, 요청마다 Access Token을 검증하고 SecurityContext에 인증 정보를 주입하도록 구현.
- Access Token과 Refresh Token을 분리 발급하고, 토큰에 권한및 사용자 ID 클레임을 포함하도록 설계.
- **Refresh Token은 Redis**에 저장하여 사용자별 최신 토큰만 유효하도록 관리하고, 재발급 시 **rotation** 및 로그아웃 시 삭제하도록 구현.
- **Redis TTL**을 적용해 만료된 토큰이 자동으로 정리되도록 구성.

로그 분석 개선

⟋ 배경

성능 저하나 응답 지연 발생 시, 단순 요청 로그나 SQL 로그만으로는 어떤 API가 왜 느린지 빠르게 파악하기 힘든 문제 발생.
특히 요청 컨텍스트와 서비스 실행 시간, 쿼리 수를 연결해서 보기 어려워 병목 원인 추적에 시간이 많이 소요됨.

⟋ 구현

- HandlerInterceptor를 활용해 요청 시작/종료 시점의 메타데이터(path, method, userId, durationMs) 로깅
- MDC를 적용해 요청 컨텍스트를 모든 로그에 일관되게 포함
- AOP(@Around) 기반으로 서비스 메서드 실행 시간 측정
- datasource-proxy와 QueryCountHolder를 활용해 쿼리 수(total/select/insert/update/delete) 집계
- MDC.clear() 적용으로 스레드 재사용 시 컨텍스트 오염 방지

감사합니다.

PHONE 010.5597.6632

MAIL kdh367036@naver.com